



第05章 （流程控制之）循环语句

张仁斌@合肥工业大学软件学院

本章主要内容



5.1 while循环



5.2 do-while循环

5.3 for循环

5.4 嵌套循环

5.5 实例研究一

5.6 关键字break和continue

5.7 实例研究二

5.8 作业与实践

引例



◆在实际应用中，经常会要求重复执行某些相同的动作，例如：

■求： $s=1+2+3+4+\dots+100$

■滚动循环显示文字信息，或输出字符串“Welcome to C++!” 100遍

◆C/C++提供循环语句（循环控制结构）控制一个或多个操作连续多次执行，例如：

```
int sum = 0;
int i = 1;
while(i <= 100)
{
    sum += i;
    i++;
}
```

```
int count = 0;
while(count < 100)
{
    cout << "Welcome to C++!\n";
    count++;
}
```

◆C/C++提供了3种循环语句：**while**循环、**do-while**循环和**for**循环

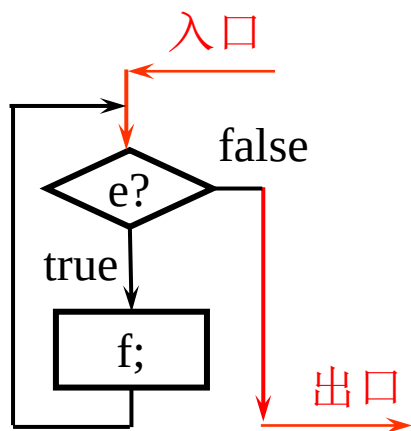


while循环的语法

◆while循环的语法为:

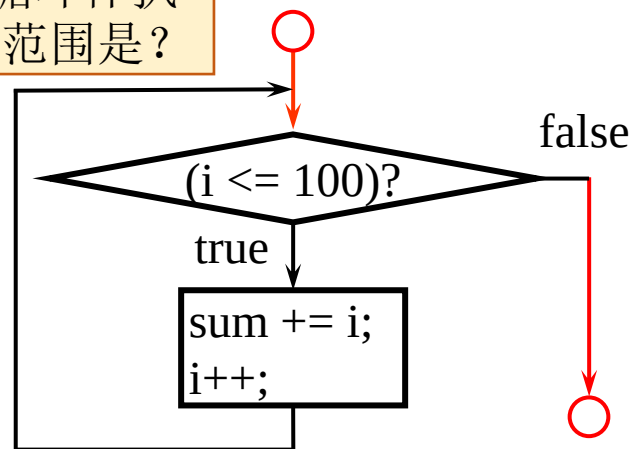
```
while (循环条件布尔表达式)
{
    语句序列; // 循环体
}
```

■当循环条件为真（true）时执行循环体，否则结束while语句（循环结束）



```
int sum = 0;
int i = 1;
while(i <= 100)
{
    sum += i;
    i++;
}
```

如果将<=改为<, 循环体执行了多少次? 累加范围是?



如果注释掉这一行，循环会结束吗?

循环控制变量i计数循环体执行次数，故此类循环称作计数控制循环，注意计数的截止条件

跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;
```

初始化count为0

```
while (count < 2)
```

```
{
```

```
    cout << "Welcome to C++!";
```

```
    count++;
```

```
}
```

跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;
```

```
while (count < 2)
```

```
{
```

```
    cout << "Welcome to C++!";
```

```
    count++;
```

```
}
```

count为0, (count < 2)为真

跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;  
while (count < 2)  
{  
    cout << "Welcome to C++!";  
    count++;  
}
```

输出Welcome to C++!

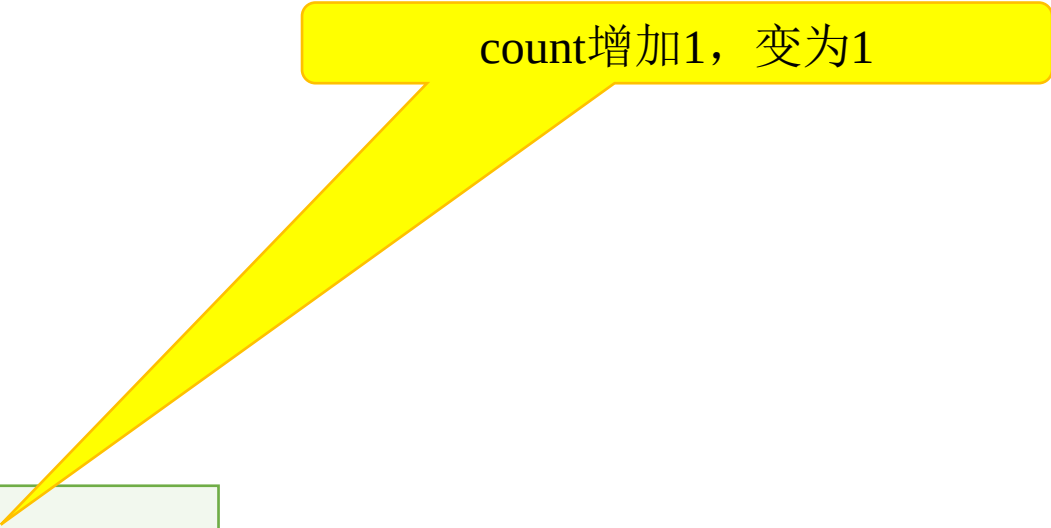
cout << "Welcome to C++!";

跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;
while (count < 2)
{
    cout << "Welcome to C++!";
    count++;
}
```

count增加1, 变为1



跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;
```

```
while (count < 2)
```

```
{
```

```
    cout << "Welcome to C++!";
```

```
    count++;
```

```
}
```

count为1, (count < 2)为真

跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;  
while (count < 2)  
{  
    cout << "Welcome to C++!";  
    count++;  
}
```

输出Welcome to C++!

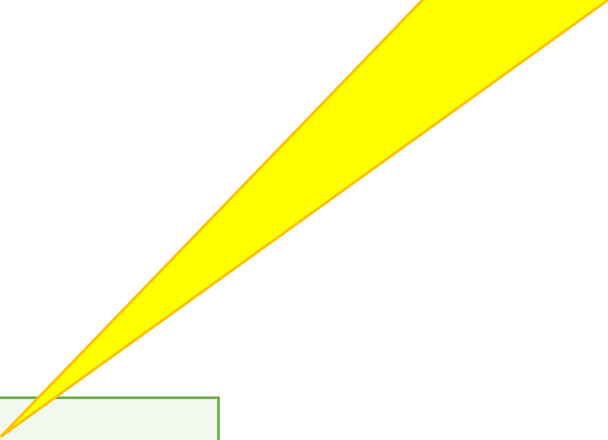
cout << "Welcome to C++!";

跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;  
while (count < 2)  
{  
    cout << "Welcome to C++!";  
    count++;  
}
```

count增加1, 变为2



跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;
```

```
while (count < 2)
```

```
{
```

```
    cout << "Welcome to C++!";
```

```
    count++;
```

```
}
```

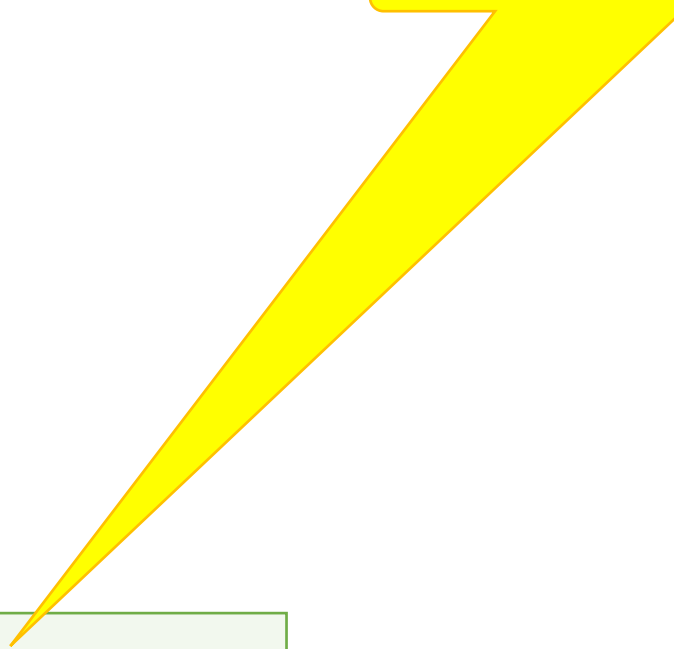
count为2, (count < 2)为假

跟踪并理解while循环的执行过程（人工推演）



```
int count = 0;
while (count < 2)
{
    cout << "Welcome to C++!";
    count++;
}
```

循环结束(退出)，执行其后语句



示例：用while循环改写减法练习程序



◆ 编写程序随机生成两个一位整数`number1`和`number2`，且`number1 >= number2`（假设这两个数分别是9和2），提示学生回答“**What is 9 – 2?**”，当学生输入答案错误时提示输入新的答案直至答案正确

■ 设计循环结构的关键是：

- 确定循环体（需重复执行的语句）
- 确定循环条件

■ 本示例的循环体应包括哪些操作？循环结束的条件是什么？

```
#include <iostream>
#include <ctime> // for time function
#include <cstdlib> // for rand and srand functions
using namespace std;
```

```
int main()
{
    // 1. Generate two random single-digit integers
    srand(time(0));
    int n1 = rand() % 10;
    int n2 = rand() % 10;

    // 2. If n1 < n2, swap them
    if (n1 < n2)
    {
        int temp = n1;
        n1 = n2;
        n2 = temp;
    }
```

```
// 3. Prompt the student to answer question
cout << "What is " << n1 << " - " << n2 << "? ";
int answer;
cin >> answer;
```

```
// 4. Repeatedly ask the user the question until it is correct
```

```
while (n1 - n2 != answer)
{
    cout << "Wrong answer. Try again. What is "
        << n1 << " - " << n2 << "? ";
    cin >> answer;
}
```

```
cout << "You got it!" << endl;
```

```
return 0;
```

```
}
```

D:\Edu-CPP\chap05\ex05-01.exe

```
What is 6 - 4? 3
Wrong answer. Try again. What is 6 - 4? 2
You got it!
```



示例：多道减法测试

◆将前面每次只出一道减法训练题的程序，改写为随机生成5道减法题，并给出答案是否正确的提示（同一道题只答题一次）

■循环体应包含哪些操作？

■结束循环的条件是什么？

D:\Edu-CPP\chap05\ex05-02.exe

```
9 - 0 = ? 9
You are correct!
4 - 1 = ? 4
Your answer is wrong.
4 - 1 should be 3
6 - 1 = ? 5
You are correct!
9 - 4 = ? 5
You are correct!
7 - 4 = ? 3
You are correct!
Correct count is 4
Test time is 78 seconds
```



```
#include <iostream>
#include <ctime>
#include <cstdlib>
using namespace std;
int main() {
    int correctCount = 0; // 正确答案计数
    int count = 0;        // 试题计数
    long startTime = time(0);
```

```
    srand(time(0));
    while (count < 5)
    {
```

如果是100道题，做了几道题就不想继续了，如何**正常**结束循环？

```
        int n1 = rand() % 10;
        int n2 = rand() % 10;
        if (n1 < n2)
        {
            int temp = n1;
            n1 = n2;
            n2 = temp;
        }
```

```
        cout << n1 << " - " << n2 << " = ? ";
        int answer;
        cin >> answer;
        if (n1 - n2 == answer)
        {
```

```
            cout << "You are correct\n";
            correctCount++;
        }
        else
            cout << "Your answer is wrong\n";
            n2 << " should be " << n1 - n2 << "\n";
```

```
        count++;
```

```
    }
```

```
    long endTime = time(0);
    long testTime = endTime - startTime;
    cout << "Correct count is " << correctCount << "\nTest time is "
        << testTime << " seconds\n";
    return 0;
}
```

增加循环控制条件，让用户确认控制循环：

```
bool isLoop = true;
while (count < 5 && isLoop)
{
    .....
    count++;
    char continueLoop;
    cout << "Enter Y to continue and N to quit: ";
    cin >> continueLoop;
    if('N' == continueLoop)
        isLoop = false;
}
```

示例：猜数字



- ◆ 编写程序随机生成0~100的整数，包括0和100，提示用户持续输入一个整数，直到该整数等于随机生成的整数。对用户的每次输入，提示输入整数太大或太小，以使用户缩小猜数的范围而猜中。
 - 猜数技巧：折半法，即第一次猜50，若提示太大，猜0~49中的25，否则猜51~100中的75，以此类推
 - 因允许用户多次猜数，直至猜中为止，故需用循环
 - 相较于减法训练程序，多了对答案的提示
 - 循环体包括猜数和对应的提示等操作

```
#include <iostream>
#include <cstdlib>
#include <ctime> // Needed for the time function
using namespace std;
int main() {
    // Generate a random number to be guessed
    srand(time(0));
    int number = rand() % 101;
```

D:\Edu-CPP\chap05\ex05-03.exe

```
Guess a magic number between 0 and 100
Enter your guess: 5
Your guess is too high

Enter your guess: 2
Your guess is too high

Enter your guess: 1
Your guess is too high

Enter your guess: 6
Your guess is too low

Enter your guess: 9
Your guess is too low

Enter your guess: 10
Yes, the number is 10
```

②或变换标记值控制循环的方式:

```
bool isLoop = true;
while (isLoop)
{
    .....
    if(-999 == guess)
        isLoop = false
    else if(guess == number)
    {
        isLoop = false;
        .....
    }
}
```

```
cout << "Guess a magic number between 0 and 100";
int guess = -1;
```

为什么要给guess赋-1?
还有别的解决方法吗?

```
while (guess != number)
{
```

```
// Prompt the user to guess the number
```

```
cout << "\nEnter your guess: ";
```

```
cin >> guess;
```

多次都猜不中时不想猜了,
如何**正常**结束循环?

```
if (guess == number)
```

```
    cout << "Yes, the number is " << number << endl;
```

```
else if (guess > number)
```

```
    cout << "Your guess is too high" << endl;
```

```
else
```

```
    cout << "Your guess is too low" << endl;
```

```
} // End of loop
```

```
return 0;
```

①增加循环控制条件, 使用标记值控制循环:

```
while (guess != number && guess != -999)
{
    .....
    cin >> guess; // 输入-999即可结束循环
    .....
}
```

示例：使用标记值控制循环



- 编程读入一组整数，计算并输出其累加和。这组整数的数量不确定，约定输入0表示输入结束

```
#include <iostream>
using namespace std;
int main() {
    int data, sum = 0;
    cout << "Enter an integer (the input ends if it is 0): ";
    cin >> data;
    while (data != 0)
    {
        sum += data;
        cout << "Enter an integer (the input ends if it is 0): ";
        cin >> data;
    }

    cout << "The sum is " << sum << endl;
    return 0;
}
```

input.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

1 2 3 4 5 6

9 0 8

D:\Edu-CPP\chap05\ex05-04.exe

```
Enter an integer (the input ends if it is 0): 2
Enter an integer (the input ends if it is 0): 3
Enter an integer (the input ends if it is 0): 4
Enter an integer (the input ends if it is 0): 0
The sum is 9
```

输入重定向: **ex05-04.exe < input.txt**

C:\WINDOWS\system32\cmd.exe

D:\Edu-CPP\chap05>ex05-04 < input.txt

```
Enter an integer (the input ends if it is 0):
Enter an integer (the input ends if it is 0):
: Enter an integer (the input ends if it is 0):
): Enter an integer (the input ends if it is 0):
0): Enter an integer (the input ends if it is 0):
s 0): Enter an integer (the input ends if it is 0):
is 0): Enter an integer (the input ends if it is 0):
is 0): The sum is 30
```

输出重定向，可以尝试：
ex05-04.exe > output.txt 或
ex05-04.exe < input.txt > output.txt

D:\Edu-CPP\chap05>



◆代码的另一种写法:

```
#include <iostream>
using namespace std;
int main() {
    bool isLoop = true;
    int data, sum = 0;
    while (isLoop)
    {
        cout << "Enter an integer (the input ends if it is 0): ";
        cin >> data;
        if(0 != data)
            sum += data;
        else
            isLoop = false;
    }
    cout << "The sum is " << sum << endl;
    return 0;
}
```

示例：读取文件中全部数据



◆编程读取文件中的全部数据，并输出其累加和

■文件中数据个数未知，且不约定结束读取的标志值

```
#include <iostream>
#include <fstream>
using namespace std;
int main() {
    ifstream input("input.txt"); // Open a file
    double number, sum = 0;

    while (!input.eof()) // Read data to the end of file
    {
        input >> number; // Read data
        cout << number << " "; // Display data
        sum += number;
    }
    input.close();
    cout << "\nTotal is " << sum << endl;
    return 0;
}
```

input.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

1 2 3 4 5 6
9 0 8

D:\Edu-CPP\chap05\ex05-05.exe

1 2 3 4 5 6 9 0 8
Total is 38

思考



下面代码中，A、B、C三处，**count<100**何时为**true**、何时为**false**？

```
int count = 0;
while(count < 100)
{
    // point A
    count << "Welcome to C++!\n";
    count++;
    // point B
}
// point C
```

下列三个循环体各循环多少次？每个循环的输出是什么？

```
int i = 1;
while(i < 10)
    if(i % 2 == 0)
        cout << i << endl;
```

```
int i = 1;
while(i < 10)
    if(i % 2 == 0)
        cout << i++ << endl;
```

```
int i = 1;
while(i < 10)
    if(i++ % 2 == 0)
        cout << i << endl;
```

下面代码的输出是什么？做出解释

```
int x = 800000000;
while(x > 0)
    x++;

cout << "x is " << x << endl;
```

本章主要内容



5.1 while循环

5.2 do-while循环



5.3 for循环

5.4 嵌套循环

5.5 实例研究一

5.6 关键字break和continue

5.7 实例研究二

5.8 作业与实践

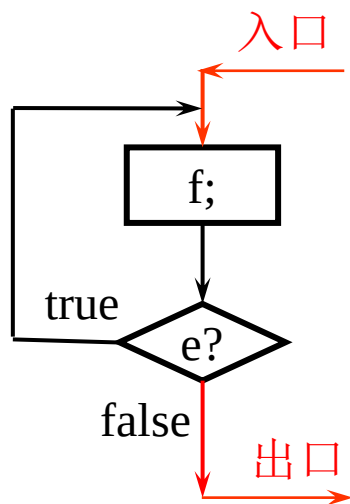


do-while循环的语法

◆ do-while循环的语法为:

```
do
{
    语句序列; // 循环体
} while (循环条件布尔表达式)
```

■ 先执行循环体，当循环条件为true时继续执行循环体，否则结束循环



■ 适用于循环体至少执行一次的场景

while与do-while区别:
while循环先判条件，后执行循环体;
do-while循环先执行循环体，后判条件

示例：用do-while循环重写while循环示例



- ◆ 编程读入一组整数，计算并输出其累加和。这组整数的数量不确定，约定输入0表示输入结束

Ex05-04

```
#include <iostream>
using namespace std;
int main() {
    int data, sum = 0;
    cout << "Enter an integer (the input ends if it is 0): ";
    cin >> data;
    while (data != 0)
    {
        sum += data;

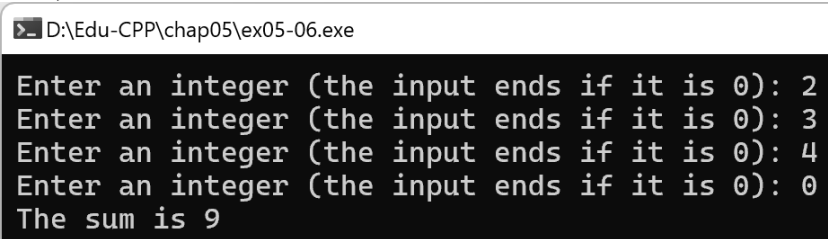
        cout << "Enter an integer (the input ends if it is 0): ";
        cin >> data;
    }

    cout << "The sum is " << sum << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;
int main() {
    int data = 0, sum = 0;
    do
    {
        sum += data;

        cout << "Enter an integer (the input ends if it is 0): ";
        cin >> data;
    }
    while (data != 0);

    cout << "The sum is " << sum << endl;
    return 0;
}
```



尝试将此前示例中的while循环改写为do-while循环

本章主要内容



5.1 while循环

5.2 do-while循环

5.3 for循环



5.4 嵌套循环

5.5 实例研究一

5.6 关键字break和continue

5.7 实例研究二

5.8 作业与实践



计数控制循环的简化

◆ 如下形式的循环

```
i = initialValue; // 初始化循环控制变量
while(i < endValue)
{
    // 循环体
    .....
    i++; // 调整循环控制变量
}
```

◆ 可以使用for循环简化（二者执行的逻辑顺序是一样的）

```
for(i = initialValue; i < endValue; i++)
{
    // 循环体
    .....
}
```



for循环语句语法

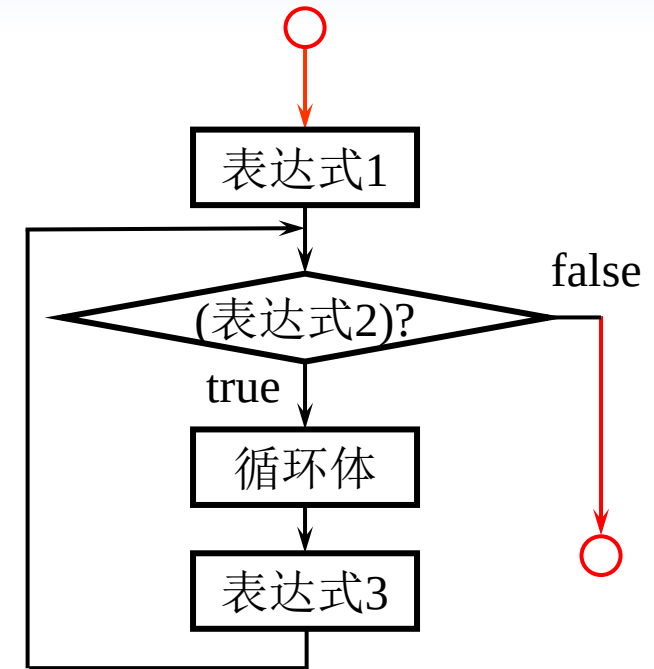
◆ 语法格式

初始赋值
先求解，**只求一次**

循环条件
为真时，执行循环体

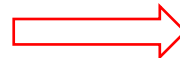
for (表达式1; 表达式2; 表达式3)
{
 语句; // 循环体
}

末尾循环体
每次执行循环体后求解



```
int sum = 0;
int i = 1;
while(i <= 100)
{
    sum += i;
    i++;
}
```

等价于



```
int sum = 0;
for(int i = 1; i <= 100; i++)
    sum += i;
```

循环控制变量只用于本循环时，最好在此定义并初始化，循环结束，变量占用的内存即释放，不能再使用

跟踪并理解for循环的执行过程（人工推演）



```
int i;
```

```
for (i = 0; i < 2; i++)
```

```
{
```

```
    cout << "Welcome to C++!";
```

```
}
```

定义变量 i

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

执行初始化，i值为0

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

i值为0, (i < 2)为真

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

输出Welcome to C++!

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

调整变量，i值为1

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

i值为1, (i < 2)为真

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

输出Welcome to C++!

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

调整变量，i值为2

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

i值为2, (i < 2)为假

跟踪并理解for循环的执行过程（人工推演）



```
int i;  
for (i = 0; i < 2; i++)  
{  
    cout << "Welcome to C++!";  
}
```

结束(退出)循环，执行其它语句



- ◆ **for (表达式1; 表达式2; 表达式3)** 中表达式1、表达式3 “内部” 可以有多个逗号分隔的语句，例如：

```
for(int i = 0, j = 0; i + j < 10; i++, j++)  
{  
    // 循环体  
    .....  
}
```

- ◆ 若**for (表达式1; 表达式2; 表达式3)**中的表达式2被省略，其值默认为**true**下面几种无限循环是等价的，建议使用最后一种

```
for( ; ; )  
{  
    // 循环体  
    .....  
}
```

省略表达式时，分号一个都不能少

```
for( ; true; )  
{  
    // 循环体  
    .....  
}
```

```
while( true )  
{  
    // 循环体  
    .....  
}
```


for循环的几种等价变化形式



◆ 求10!

```
int main()
{
    int i, s;
    for(i=1, s=1; i <= 10; i++)
        s = s * i;
    cout << "10!=" << s;
    return 0;
}
```

for (表达式1; 表达式2; 表达式3)中任意一个或者两个甚至全部表达式可以省略，但分号 (;) 不能省略

表达式1：用于进入循环体之前给某些变量赋初值，若省略，可在for语句前给变量赋初值

表达式2：循环条件，若省略，需在循环体中增加中断循环的语句，否则为无限循环（死循环）

表达式3：每次循环体结束后对某些变量进行修改，若省略，可在循环体内对变量进行修改

```
i = 1;
s = 1;
for( ; i <= 10; i++)
    s = s * i;
```

分号不能省

```
for(i = 1, s = 1; ; i++)
{
    s = s * i;
    if(i < 10) break;
}
```

分号不能省

```
s = 1;
for(i = 1; i <= 10; ; )
{
    s = s * i;
    i++;
}
```

分号不能省



思考

◆ 下面两个循环的结果sum1、sum2值是否相同？为什么？

```
int sum1 = 0;
```

```
for(int i = 0; i < 10; ++i)  
    sum1 += i;
```

```
int sum2 = 0;
```

```
for(int i = 0; i < 10; i++)  
    sum2 += i;
```

```
D:\Edu-CPP\chap05\ex05-07.exe
```

```
The sum1 is 45  
The sum2 is 45
```

循环语句的选择



◆ 若循环次数已知，优先选用for循环；若循环次数未知，且循环体至少执行一次，优先选用do-while循环；三种循环可以相互之间等价转换

```
#include <iostream>
using namespace std;
int main() {
    bool isLoop = true;
    int data, sum = 0;
    while (isLoop)
    {
        cout << "Enter an integer (the input ends if it is 0): ";
        cin >> data;
        if(0 != data)
            sum += data;
        else
            isLoop = false;
    }
    cout << "The sum is " << sum << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;
int main() {
    bool isLoop = true;
    int data, sum = 0;
    do
    {
        cout << "Enter an integer (the input ends if it is 0): ";
        cin >> data;
        if(0 != data)
            sum += data;
        else
            isLoop = false;
    } while (isLoop);
    cout << "The sum is " << sum << endl;
    return 0;
}
```

识别和修改代码错误



```
int main()
{
    for(int i = 0; i < 10; i++);
        sum += i;

    if(i < j);
        cout << i << endl;
    else
        cout << j << endl;

    while(j < 10);
    {
        j++;
    }

    do {
        j++;
    }
    while(j < 10)
}
```

本章主要内容



5.1 while循环

5.2 do-while循环

5.3 for循环

5.4 嵌套循环



5.5 实例研究一

5.6 关键字break和continue

5.7 实例研究二

5.8 作业与实践



◆ 嵌套循环由一个外层循环和一个或多个内层循环组成，外层循环每迭代一次，都会重入内层循环，开始一次新的执行

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main()
{
```

```
    cout << "    乘法口诀表\n";
    cout << "-----\n";
```

```
    // Display the number title
```

```
    cout << " |";
    for (int j = 1; j <= 9; j++)
        cout << setw(3) << j;
```

```
    cout << "\n";
```

```
    // Display table body
    for (int i = 1; i <= 9; i++)
```

```
    {
```

```
        cout << i << " |";
        for (int j = 1; j <= 9; j++)
```

```
        {
```

```
            // Display the product and align properly
```

```
            cout << setw(3) << i * j;
```

```
        }
```

```
        cout << "\n";
```

```
    }
```

```
    return 0;
```

```
}
```

该语句执行了多少次?

D:\Edu-CPP\chap05\ex05-08.exe

乘法口诀表

	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81



◆ 内外循环**独立**执行

```
for(int j=0; j<10; j++)  
{  
    for(int i=0; i<10; i++)  
        cout << '*' << " ";  
    cout << endl;  
}
```

◆ 内部循环**依赖于**外部循环

```
for(int j=0; j<10; j++)  
{  
    for(int i=0; i<j+1; i++)  
        cout << '*' << " ";  
    cout << endl;  
}
```

D:\Edu-CPP\chap05\ex05-09.exe

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

```
*  
* *  
* * *  
* * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

本章主要内容



5.1 while循环

5.2 do-while循环

5.3 for循环

5.4 嵌套循环

5.5 实例研究一



5.6 关键字break和continue

5.7 实例研究二

5.8 作业与实践



示例1：求最大公约数

◆ 编写程序提示用户输入两个正整数，计算并输出其最大公约数

```
#include <iostream>
using namespace std;
int main() {
    int n1, n2, gcd = 1, k = 2;
    cout << "请输入两个正整数: ";
    cin >> n1 >> n2;
```

```
    for(int k=2; k <= n1 && k <= n2; k++)
        if (n1 % k == 0 && n2 % k == 0)
            gcd = k;
```

```
    if(n1 <= 0 || n2 <= 0)
        cout << "输入数据无效! " << endl;
    else
    {
```

```
        while (k <= n1 && k <= n2)
```

```
        {
```

```
            if (n1 % k == 0 && n2 % k == 0)
```

```
                gcd = k;
```

```
            k++;
```

```
        }
```

```
        cout << n1 << "和" << n2 << "的最大公约数是" << gcd << endl;
    }
```

```
    return 0;
```

```
}
```

D:\Edu-CPP\chap05\ex05-10.exe

请输入两个正整数：125 2525
125和2525的最大公约数是25



示例2：预测未来的学费

◆ 假设某学校今年的学费是10000美元，学费每年递增7%，多少年后学费会翻倍？

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    double tuition = 10000; // Year 0
    int year = 0;
```

```
    while (tuition < 20000)
```

```
    {
```

```
        tuition = tuition * 1.07;
```

```
        year++;
```

```
    }
```

```
    cout << "Tuition will be doubled in " << year << " years" << endl;
    cout << setprecision(2) << fixed << showpoint << "Tuition will be $"
        << tuition << " in " << year << " years" << endl;
```

```
    return 0;
```

```
}
```

可有其它解法？ $10000 * 1.07^{\text{year}} \geq 20000$,
 $\text{year} \geq \log_{10}(20000/10000) / \log_{10}(1.07)$,
 $\text{year} = \text{ceil}(\log_{10}(2) / \log_{10}(1.07)) = \text{ceil}(10.2)$

D:\Edu-CPP\chap05\ex05-11.exe

```
Tuition will be doubled in 11 years
Tuition will be $21048.52 in 11 years
```

示例3：十进制转十六进制



- ◆编程提示用户输入一个十进制整数，将其转换成十六进制数字字符串
 - 辗转相除法

(回顾)



◆ 十进制转二进制

十进制整数转化为其它进制的方法类似（不作要求）

■ 整数：辗转相除法

■ 纯小数：与2相乘后取整数部分

$$\begin{array}{r|l} 2 & 34 \\ \hline 2 & 17 \text{ --- } 0 \text{ 低位} \\ 2 & 8 \text{ --- } 1 \\ 2 & 4 \text{ --- } 0 \\ 2 & 2 \text{ --- } 0 \\ 2 & 1 \text{ --- } 0 \\ 0 & \text{ --- } 1 \text{ 高位} \end{array} \quad \begin{array}{c} \uparrow \\ \uparrow \\ \uparrow \\ \uparrow \\ \uparrow \end{array} \quad \Rightarrow \quad 34_{10} = 100010_2$$

每次相乘后去掉整数部分，不断乘下去，直到小数部分为0或达到指定的精度为止，然后取每次相乘后的整数部分即可

绝大部分浮点数无法用二进制精确表示，如0.1, 0.2, 0.3, ...

拓展阅读：计算机中浮点数的表示和运算——IEEE 浮点运算标准

$$\begin{array}{rcl} 0.3125 \times 2 & = & 0.625 \\ 0.625 \times 2 & = & 1.25 \\ 0.25 \times 2 & = & 0.5 \\ 0.5 \times 2 & = & 1.0 \end{array} \quad \begin{array}{c} \downarrow \\ \downarrow \\ \downarrow \\ \downarrow \end{array} \quad \Rightarrow \quad 0.3125_{10} = 0.0101_2$$



```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    // Prompt the user to enter a decimal integer
    cout << "Enter a decimal number: ";
    int decimal;
    cin >> decimal;

    // Convert decimal to hex
    string hex = "";
```

```
while (decimal != 0)
{
    int hexValue = decimal % 16;

    // Convert a decimal value to a hex digit
    char hexChar = (hexValue <= 9 && hexValue >= 0) ?
        static_cast<char>(hexValue + '0') :
        static_cast<char>(hexValue - 10 + 'A');

    hex = hexChar + hex;
    decimal = decimal / 16;
}

cout << "The hex number is " << hex << endl;

return 0;
}
```

D:\Edu-CPP\chap05\ex05-12.exe

```
Enter a decimal number: 1234
The hex number is 4D2
```

本章主要内容



5.1 while循环

5.2 do-while循环

5.3 for循环

5.4 嵌套循环

5.5 实例研究一

5.6 关键字break和continue

5.7 实例研究二

5.8 作业与实践





◆ C/C++提供了break和continue两个关键字（语句），为循环语句提供额外的控制

- 可以在循环中使用break终止循环，使用continue语句跳过循环体中剩余的语句（循环体“短路”）
- switch语句也可以使用break语句终止switch语句

break应用示例



```
#include <iostream>
using namespace std;

int main() {
    int sum = 0;
    int number = 0;

    while (number < 20)
    {
        number++;
        sum += number;
        if (sum >= 100)
            break;
    }

    cout << "The number is " << number << endl;
    cout << "The sum is " << sum << endl;

    return 0;
}
```

D:\Edu-CPP\chap05\ex05-13.exe

```
The number is 14
The sum is 105
```

注释掉黄颜色标注的两行代码时的程序输出：

D:\Edu-CPP\chap05\ex05-13.exe

```
The number is 20
The sum is 210
```


continue应用示例



```
#include <iostream>
using namespace std;

int main()
{
    int sum = 0;
    int number = 0;

    while (number < 20)
    {
        number++;
        if (number == 10 || number == 11)
            continue;
        sum += number;
    }

    cout << "The sum is " << sum << endl;

    return 0;
}
```

D:\Edu-CPP\chap05\ex05-14.exe

The sum is 189

break的替代



◆ **break**、**continue**语句可以被其它语句替代，但**break**、**continue**使程序更简洁

```
int factor = 2;
while (factor <= n)
{
    if(n % factor == 0)
        break;
    factor++;
}
cout << n << "的除1以外的最小因子是" << factor << endl;
```

不使用**break**的等价代码:

```
bool found = false;
int factor = 2;
while (factor <= n && !found)
{
    if(n % factor == 0)
        found = true;
    else
        factor++;
}
cout << n << "的除1以外的最小因子是" << factor << endl;
```



◆ 在嵌套循环中，**break**、**continue**语句仅退出或“短路”包含该语句的当前层循环

```
for (int i = 1; i < 4; i++)  
{  
    for (int j = 1; j < 4; j++)  
    {  
        if (i * j > 2)  
            break;  
        cout << i * j << endl;  
    }  
    cout << i << endl;  
}
```

```
for (int i = 1; i < 4; i++)  
{  
    for (int j = 1; j < 4; j++)  
    {  
        if (i * j > 2)  
            continue;  
        cout << i * j << endl;  
    }  
    cout << i << endl;  
}
```

本章主要内容



5.1 while循环

5.2 do-while循环

5.3 for循环

5.4 嵌套循环

5.5 实例研究一

5.6 关键字break和continue

5.7 实例研究二

5.8 作业与实践

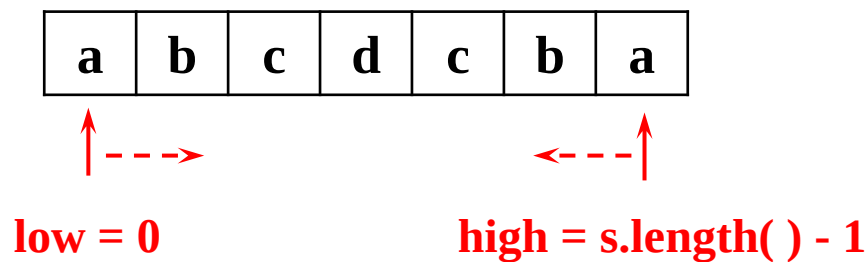


示例：检查回文



◆ 如果一个字符串按照正序和逆序读都是相同的，则该字符串是回文，例如mom、dad、noon。编写程序检查一个字符串是否是回文。

■ 算法：检查字符串的第一个字符和最后一个字符是否相同；若相同，则检查第二个字符和倒数第二个字符知否县共同；持续该检查过程，直至发现不相同或全部字符都被检查过了（除了奇数个数的字符最中间的字符）



```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    // Prompt the user to enter a string
    cout << "Enter a string: ";
    string s;
    getline(cin, s);

    // The index of the first character in the string
    int low = 0;

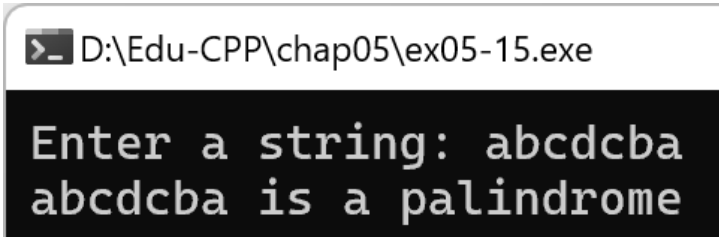
    // The index of the last character in the string
    int high = s.length() - 1;
```

```
bool isPalindrome = true;
while (low < high)
{
    if (s[low] != s[high])
    {
        isPalindrome = false; // Not a palindrome
        break;
    }

    low++;
    high--;
}

if (isPalindrome)
    cout << s << " is a palindrome" << endl;
else
    cout << s << " is not a palindrome" << endl;

return 0;
}
```



D:\Edu-CPP\chap05\ex05-15.exe

Enter a string: abdcba
abdcba is a palindrome

示例：输出素数



◆ 如果一个大于1的整数，其正因数只有1及其自身，则该数是一个素数。为了检查number是否是素数，可以检查它是否能被2、3、4、...、number/2整除，若存在因数（被整除），则number不是素数，否则是素数。编写程序输出自然数中**前50个**素数，以阵列形式显示，**每行显示10个素数**

- 输出10个素数后执行换行
- 每列宽度统一

```
D:\Edu-CPP\chap05\ex05-16.exe

The first 50 prime numbers are
  2   3   5   7  11  13  17  19  23  29
 31  37  41  43  47  53  59  61  67  71
 73  79  83  89  97 101 103 107 109 113
127 131 137 139 149 151 157 163 167 173
179 181 191 193 197 199 211 223 227 229
```

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    const int NUMBER_OF_PRIMES = 50;
    const int NUMBER_PER_LINE = 10;
    int count = 0; // Count the number of prime numbers
    int number = 2; // A number to be tested for primeness
    cout << "The first 50 prime numbers are \n";
    // Repeatedly find prime numbers
    while (count < NUMBER_OF_PRIMES)
    {
        bool isPrime = true; // Is the current number prime?
        // Test if number is prime
        for (int divisor = 2; divisor <= number / 2; divisor++)
        {
            if (number % divisor == 0)
            {
                // If true, the number is not prime
                isPrime = false;
                break; // Exit the for loop
            }
        }
    }
}
```

```
// Display the prime number and increase the count
if (isPrime)
{
    count++; // Increase the count

    if (count % NUMBER_PER_LINE == 0)
        // Display the number and advance to the new line
        cout << setw(4) << number << endl;
    else
        cout << setw(4) << number;
}

// Check if the next number is prime
number++;
}

return 0;
}
```


本章主要内容



5.1 while循环

5.2 do-while循环

5.3 for循环

5.4 嵌套循环

5.5 实例研究一

5.6 关键字break和continue

5.7 实例研究二

5.8 作业与实践



第5章作业



(1)编写程序读入整数（数目未定），统计读入正数和负数的数量，并计算这些整数的总和与平均值（0不计算在内），如果读入0，程序即终止。平均值以浮点数显示

(2)输出如下列表，显示0~360度中以10度为单位增长的度数对应的sin值与cos值，精确到小数点后4位

Degree	Sin	Cos
0	0.0000	1.0000
10	0.1736	0.9848
.....		
350	-0.1736	0.9848
360	0.0000	1.0000

(3)编写程序提示用户输入学生数（大于等于3）和每个学生的姓名及成绩，输出成绩排第一和第二的学生的姓名和成绩



- (4)编写程序输出100~200之间全部能被5和6之一整除的，且只能被二者之一整除的整数，每行显示10个，数字间用空格分隔
- (5)编写程序读入一个整数，由小到大显示其全部因子。例如，如果输入整数为120，输出应该是：2 2 2 3 5
- (6)编写程序提示用户输入1~15中的某个数字，输出金字塔图案。例如，输入7时，图案为：

```
      1
    2 1 2
  3 2 1 2 3
4 3 2 1 2 3 4
5 4 3 2 1 2 3 4 5
6 5 4 3 2 1 2 3 4 5 6
7 6 5 4 3 2 1 2 3 4 5 6 7
```



(7)编写程序计算下面级数的和

$$\frac{1}{3} + \frac{3}{5} + \frac{5}{7} + \frac{7}{9} + \frac{11}{13} + \cdots + \frac{95}{97} + \frac{97}{99}$$

(8)若一个正整数等于它的全部正因子（不包括它本身）之和，则该正整数称作完全数。例如，6为第一个完全数，因为 $6=3+2+1$ ，下一个为 $28=14+7+4+2+1$ 。小于10000的完全数有4个，编写程序找出这4个数字

(9)编写程序提示用户输入一个字符串，输出字符串中大写字母的个数

(10)编写程序提示用户输入字符串，以相反顺序输出字符串。例如输入“ABC”，输出“CBA”

(11)编写程序，计算名为countletters.txt文件中字母的个数

(12)将字母A、E、I、O、U及其小写定为元音。编写程序提示用户输入字符串，输出字符串中元音和辅音的个数